

Project Synapse: Org-Wide Zero-ETL Semantic Data Core

UNIFIED MASTER ARCHITECTURAL SPECIFICATION & IMPLEMENTATION ROADMAP

0. Abstract: Architectural Completeness Goal

Project Synapse demonstrates architectural feasibility for an enterprise-wide, discrepancy-tolerant, schema-on-read context platform. Instead of verifying petabyte-scale compute arrays, this document guarantees production completeness by addressing structural vulnerabilities—such as deterministic parsing, inference token blowouts, dynamic access controls, and query latency—with concrete software primitives.

1. The Problem Statement: The Schema Bottleneck

Modern digital systems process highly transactional, chaotic, and heterogeneous workloads (OLTP). To make this information actionable for analytical engines, organizations traditionally enforce a strict **Schema-on-Write** rule. This mechanism incurs heavy operations overhead: multi-tiered ETL pipelines must pull, format, deduplicate, and forcefully normalize data before landing it into tables.

This paradigm enforces an artificial, premature agreement across distinct domains that inherently view data differently. Upstream production shifts break downstream pipelines, batch latency delays execution, and harsh normalization pipelines strip away the critical anomalies and context that reasoning engines depend on to resolve nuanced problems.

The Synapse Core Thesis:

Store raw operational telemetry permanently alongside rich lineage tags. Defer structural layout and validation logic completely to query time via a schema-on-read execution model. Maintain an evolving graph of entities, facts, and raw conflicts rather than maintaining a single cleaned database as the singular source of absolute truth.

2. Open-Source Ecosystem Components (Engine Pieces)

To completely remove conventional ETL overhead, Project Synapse orchestrates four specialized open-source core frameworks:

1. Graphiti (by Zep)

github.com/getzep/graphiti

Enterprise Stack Role: Continuous Context & Temporal Relation Modeler.

Utilizes a Learned Ontology model to parse data sequentially, dynamically indexing evolving entity connections and temporal states over time without manual schema declarations.

Defeats the lack of temporal context; autonomously tracks entity states as they shift across disparate logs.

2. GraphRAG (by Microsoft Research)

github.com/microsoft/graphrag

Enterprise Stack Role: Hierarchical Abstractor & Global Query Synthesizer.

Clusters unstructured text segments into hierarchical summaries, allowing global thematic queries over wide document corpora where localized vector lookups typically fail.

Addresses global systems synthesis; provides cross-domain abstractions without structured predefined schemas.

3. Data-Juicer (by Alibaba Tongyi Lab)

github.com/datajuicer/data-juicer

Enterprise Stack Role: Multi-Format Ingestion Pipeline & Raw Data Streamliner.

Provides a matrix of sandboxed data operators to format, deduplicate, and parse chaotic streams (JSON records, raw telemetry, text) into stable inputs for model processing.

Solves raw byte normalization; formats dirty streams at runtime without enforcing rigid destination target schemas.

4. PageIndex (by VectifyAI)

github.com/VectifyAI/PageIndex

Enterprise Stack Role: Reasoning-Based Document & Structural Tree Navigator.

Bypasses high-cost dense vector database architectures by building layout-aware coordinate indexes directly on documents, enabling agents to navigate to structural text positions dynamically.

Mitigates dense vector monoculture scaling boundaries; uses raw layout trees for fast, highly targeted retrieval.

3. The Control Plane: Mathematical Guardrails & Specifications

3.1 Mathematical Token Budgeting & Routing

To prevent large context query windows from causing cost escalations or server execution timeouts, the query routing subsystem evaluates an explicit Information Density Factor (IDF) dynamically over incoming chunks. The

routing optimization engine minimizes costs (\$V_c\$) and latency (\$V_l\$) while ensuring the confidence metric (\$C_f\$) tracks above an enterprise target threshold (\$T\$):

Minimize: $\alpha V_c + \beta V_l$ **subject to** $C_f \geq T$

The Information Density Factor (IDF) evaluates how structurally dense the extracted information is relative to its raw token payload footprint:

$$IDF = (\text{Count of Evaluated Predicates}) / (\text{Total Token Count})$$

- **High Density (IDF ≥ 0.75):** Routed entirely away from long-context LLMs, executing via localized, fast cross-encoder reranking blocks.
- **Low Density (IDF < 0.25):** Routed via PageIndex structural layout maps, isolating token depth exclusively to specific leaf indices.

3.2 Operational Conflict Resolution Matrix

Project Synapse does not filter out conflicting data records at write time. Instead, it computes a dynamic Validity Weight (\$W_v\$) at the query layer using Authority Rank (\$A_r\$), Lineage Proximity (\$L_p\$), and a Temporal Decay coefficient (\$\lambda\$):

$$W_v = (A_r \times e^{-\lambda \cdot \Delta t}) + L_p$$

Conflict Typology	Technical Core Challenge	System Behavior
Scalar Clash	Opposing numeric metrics reported by distinct operational nodes inside the same time frame.	Flags as AMBIGUOUS_CONFLICT; returns both values with respective source lineages if no explicit human_pin is set.
Temporal Supersession	Downstream CDC events reporting conflicting states over an identical entity instance.	If the newer event scores a higher \$W_v\$, the current view shifts state while maintaining the past state as a historic edge.
Domain Overlap	Opposing attribute states claimed across cross-functional domain barriers (e.g., CRM vs Billing).	Resolves via primary authority maps inside the Ontology Registry; labels alternative fields as hidden aliases.

3.3 Cryptographic ABAC Propagation

Unstructured storage blocks carry an immutable Attribute-Based Access Control (ABAC) manifest block. When summaries or facts are derived by reasoning nodes, the engine applies a mathematical intersection of source rights:

$$\text{Derived Fact Key} = \text{ACL}_{\text{Object 1}} \cap \text{ACL}_{\text{Object 2}} \cap \dots \cap \text{ACL}_{\text{Object n}}$$

If a querying user's authorization manifest fails to match the evaluated derived fact token key, the node drops from the model attention window before prompt compilation.

4. Core Schema Specifications

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "SynapseDataCore",
  "defs": {
    "RawObject": {
      "type": "object",
      "required": ["object_id", "source_system", "content_hash", "ingested_at", "bytes_ref", "acl_tags"],
      "properties": {
        "object_id": { "type": "string", "format": "uuid" },
        "source_system": { "type": "string" },
        "content_hash": { "type": "string" },
        "ingested_at": { "type": "string", "format": "date-time" },
        "bytes_ref": { "type": "string", "format": "uri" },
        "acl_tags": { "type": "array", "items": { "type": "string" } }
      }
    },
    "Fact": {
      "type": "object",
      "required": ["fact_id", "subject_entity_id", "predicate", "object", "confidence", "evidence_refs"],
      "properties": {
        "fact_id": { "type": "string", "format": "uuid" },
        "subject_entity_id": { "type": "string", "format": "uuid" },
        "predicate": { "type": "string" },
        "object": { "type": "string" },
        "confidence": { "type": "number", "minimum": 0, "maximum": 1 },
        "evidence_refs": { "type": "array", "items": { "type": "string", "format": "uuid" } }
      }
    },
    "Conflict": {
      "type": "object",
      "required": ["conflict_id", "subject_entity_id", "predicate", "competing_fact_ids", "status"],
      "properties": {
        "conflict_id": { "type": "string", "format": "uuid" },
        "subject_entity_id": { "type": "string", "format": "uuid" },
        "predicate": { "type": "string" },
        "competing_fact_ids": { "type": "array", "items": { "type": "string", "format": "uuid" } },
        "status": { "type": "string", "enum": ["open", "resolved", "accepted_plural"] }
      }
    }
  }
}
```

5. Prototype Testing Path: Infrastructure & Operational Incidents

The ideal validation landscape for Project Synapse is **Infrastructure & Operational Incidents**. This domain naturally produces chaotic, conflicting data profiles where systems disagree continuously during an incident event (e.g., automated deployment servers reporting clean build metrics while Kubernetes clusters output loop crashes and engineering groups flag rollbacks manually inside chat rooms).

By building the initial playground within this vertical, we demonstrate how schema-on-read systems successfully reconstruct complex incidents without requiring expensive data synchronization layers.

6. Implementation Roadmap

- **Phase 0:** Architecture ADR sign-off, schema locking, and validation mockups.
- **Phase 1:** Data-Juicer ingestion script deployment tracking raw streams into an object store.
- **Phase 2:** Graphiti instance setup mapping out dynamic entities and conflict flags on read.
- **Phase 3:** Front-end query UI deployment on Vercel utilizing Google AI Studio long-context engines.